Energy Efficiency of LLM-Generated Code: A Cross-Language and Cross-Platform Empirical Study

Brisild Capani
[TBD] Affiliation
brisild.capani@aeb.com

October 25, 2026

Abstract

TODO: Translate and condense the existing German abstract (~300 words) to ~200 words. Final numbers and findings can only be added after the experiments; until then, the abstract describes goal and method only.

Large Language Models (LLMs) are increasingly used to generate source code, yet their evaluation focuses predominantly on functional correctness. The energy consumed when the generated code is *executed* has received comparatively little attention. This paper presents an empirical study of the energy efficiency of LLM-generated code along three factors: the generating LLM, the target programming language (Python, Java, C++), and the hardware platform on which the code runs (desktop PC, Raspberry Pi, server). Using a full factorial design over [TBD] programming tasks, we measure energy consumption, execution time, and peak memory usage of functionally correct solutions and analyse main and interaction effects with mixed-effects models. In particular, we investigate whether the relative ranking of LLMs with respect to energy efficiency is stable across languages and platforms.

TODO: Results and implications.

1 Introduction

LLM-based assistants have become part of everyday software development, generating anything from small snippets to complete functions. Benchmarks for code generation predominantly measure *functional correctness*, e.g., the share of generated solutions that pass a set of unit tests. Non-functional properties such as runtime performance and, in particular, energy efficiency are rarely considered, even though generated code may be executed many times in production and on a wide range of devices, from data-centre servers to resource-constrained embedded systems.

TODO: Add 1–2 citations on the ICT/software share of energy consumption and on the adoption of AI coding assistants.

Early studies indicate that the energy efficiency of LLM-generated code varies between models and often falls short of human-written reference solutions [1, 6, 15]. However, existing studies typically vary only a subset of the factors that plausibly influence energy consumption: they either cover several languages on a small number of similar platforms [15], or several heterogeneous platforms for a single language [1]. It therefore remains unclear whether a statement such as "model A generates more energy-efficient code than model B" holds in general, or only for a particular language and execution environment.

This paper addresses this question with a full factorial experiment in which [TBD] LLMs generate solutions for [TBD] programming tasks in Python, Java, and C++. All functionally correct solutions are executed on three heterogeneous platforms (an x86 desktop PC, an x86

server, and an ARM-based Raspberry Pi), and their energy consumption, execution time, and memory usage are measured under controlled conditions. Our central research question is:

> *To what extent do the choice of LLM, programming language, and hardware platform influence the energy efficiency of LLM-generated code?*

The contributions of this paper are:

- A full factorial empirical study of LLM-generated code across [TBD] LLMs, three programming languages, and three hardware platforms.
- A statistical analysis of main *and interaction* effects, with a particular focus on whether the energy-efficiency ranking of LLMs is stable across languages and platforms.
- An analysis of the relationship between energy, execution time, and memory for generated code.
- A replication package containing tasks, prompts, generated code, measurement scripts, and raw data.

TODO: Paper outline paragraph ("The remainder of this paper is structured as follows...") once the structure is final.

2 Background

2.1 LLM-Based Code Generation

TODO: Short overview: how LLMs generate code, common benchmarks (HumanEval, MBPP, EvalPlus, EvoEval, LeetCode-based sets), pass@k as correctness metric. Cite originals.

2.2 Software Energy Measurement

TODO: Energy vs. power ($E = \int P\,dt$); software-based estimation (Intel/AMD RAPL, tools building on it such as Scaphandre, CodeCarbon, perf) vs. hardware-based measurement (external power meters, INA219/INA226 shunt sensors). Explain which components each method captures (package, DRAM, whole system) and why RAPL is not available on ARM-based Raspberry Pi boards.

2.3 Energy Efficiency of Programming Languages

Pereira et al. [10] compared 27 programming languages on the Computer Language Benchmarks Game and examined how energy, time, and memory relate.

TODO: Summarise key methodological points relevant to our study (compiled vs. virtual-machine vs. interpreted languages; faster is not always more energy-efficient) – in own words, without anticipating our results.

3 Related Work

3.1 Energy Efficiency of LLM-Generated Code

Single-language studies. Most empirical work on the energy efficiency of LLM-generated code targets Python on a single platform. Islam et al. [6] compare code generated by 20 LLMs for 878 LeetCode problems with canonical human solutions on an Intel Xeon server, using RAPL via `perf`, and find LLM solutions to be less energy-efficient on average, with large differences between models and algorithmic categories. Cappendijk et al. [3] and Ashraf et al. [2] study the effect of prompt modifications and of model size, respectively; both report that outcomes depend on the specific combination of model and problem. Vartziotis et al. [16] propose a "green capacity" metric for comparing code assistants.

Hardware platforms. Cursaru et al. [4] measure code generated by a single model (Code Llama) for C++, JavaScript, and Python on a Raspberry Pi with an external power monitor and observe that efficiency depends strongly on language and problem. Apsan et al. [1] are, to our knowledge, the only study to execute LLM-generated code on three heterogeneous platforms (an Intel Xeon server, an Intel Core i9 PC, and an ARM Cortex-A72 Raspberry Pi), comparing six LLMs and four prompting techniques against human and expert-written Python code. Each platform is analysed separately, using aligned rank transform (ART) ANOVA with LLM and solution type as factors. The authors observe that the most efficient LLM differs between platforms, but this observation is not tested statistically, and the study is restricted to Python and nine problems.

Multiple languages. Solovyeva et al. [15] evaluate three LLMs (GitHub Copilot, GPT-4o, o1-mini) on 53 hard LeetCode problems in Python, Java, and C++ on an Apple M3 laptop and an Intel PC, measuring CPU energy with `powermetrics` and RAPL. They use pairwise Mann–Whitney U tests and Spearman correlations between platforms; C++ programs are compiled without optimisation (`-O0`). Interactions between model, language, and platform are not modelled. Rekhi et al. [14] evaluate eight prompting strategies across ten LLMs for Python and C++ on a single Intel cloud platform and note qualitatively that the effect of prompting is inconsistent across models and languages, without inferential statistics.

Secondary studies. A recent systematic literature review of 18 primary studies [8] concludes that evidence is "limited and highly heterogeneous", that most studies evaluate generated code on a single hardware configuration, that few use direct hardware-based measurement, and explicitly calls for cross-platform evaluations.

3.2 Runtime Efficiency Benchmarks

A parallel line of work proposes benchmarks for the runtime efficiency (execution time and memory, not energy) of generated code, e.g., EffiBench [5], EvalPerf [7], ENAMEL [13], and COFFE [9]. EffiBench-X [12] extends this to six languages, including Python, Java, and C++, and reports language-specific differences in the relative efficiency of LLMs. These benchmarks are candidate task sources for our study (Section 5), but they measure on a single platform and do not report energy.

3.3 Energy Efficiency of Programming Languages

Pereira et al. [10, 11] rank up to 27 languages by energy, time, and memory on the Computer Language Benchmarks Game and show that the fastest language is not always the most energy-efficient. Their results concern human-written, highly optimised programs on a single x86 machine; whether similar relations hold for LLM-generated code and across architectures is open.

3.4 Research Gap

Table 1 summarises the closest studies. Three observations motivate our work:

1. *No study crosses multiple languages with architecturally heterogeneous platforms.* Multi-language studies use one or two x86/Apple platforms [14, 15]; the only three-platform study is Python-only [1].

2. *Interaction effects are not tested.* Several studies report that results "depend on" model, language, or platform [1, 3, 4, 14], but none models these dependencies as interaction terms or tests the stability of LLM rankings.

Table 1: Closest empirical studies on the energy efficiency of LLM-generated code. Ext. = external power meter; RAPL = software-based (Intel RAPL).

Study	LLMs	Languages	Platforms	Measurement	Interact.
Cursaru et al. [4]	1	C++, JS, Py	RPi	Ext.	[TBD]
Cappendijk et al. [3]	5	Py	1 (x86)	RAPL	no
Islam et al. [6]	20	Py	1 (x86 server)	RAPL	no
Solovyeva et al. [15]	3	Py, Java, C++	Mac (M3), PC (x86)	powermetrics, RAPL	no
Apsan et al. [1]	6	Py	Server, PC, RPi	EnergiBridge, Ext.	partly[a]
Rekhi et al. [14]	10	Py, C++	1 (x86 cloud)	RAPL	no
This study	[TBD]	Py, Java, C++	PC, Server, RPi	[TBD]	yes

[a] LLM × prompt interaction tested within each platform; no interaction with platform.

3. *Measurement and statistics are heterogeneous* [8]: several studies report averages without inferential statistics, and C++ is sometimes compiled without optimisation.

Our study addresses these points with a full factorial design (LLM × language × platform), a mixed-effects analysis with explicit interaction terms, and a test of rank stability across languages and platforms.

TODO: Verify Cappendijk et al. platform/tool and Cursaru et al. statistics from full texts. Re-run the arXiv/Scopus search shortly before submission.

4 Research Questions and Hypotheses

RQ1 Which LLMs generate the most energy-efficient code for identical programming tasks?

RQ2 How does the energy efficiency of generated code differ between Python, Java, and C++?

RQ3 How do energy consumption and execution time of generated code change across a desktop PC, a Raspberry Pi, and a server?

RQ4 Are there statistically significant interaction effects between LLM, programming language, and hardware platform, i.e., does the ranking of LLMs change depending on language and platform?

RQ5 How are execution time, energy consumption, and memory usage related?

Each research question is accompanied by a hypothesis (null hypotheses $H_{x,0}$ state the absence of the effect). All hypotheses are tested; none is presupposed to hold.

H1 (RQ1)
Energy consumption of generated code differs between LLMs (main effect of LLM).

H2 (RQ2, RQ4)
The effect of the LLM on energy consumption depends on the programming language (LLM × language interaction).

H3 (RQ3, RQ4)
The effect of the LLM on energy consumption depends on the hardware platform (LLM × platform interaction).

H4 (RQ4)
There is a three-way interaction LLM × language × platform.

H5 (RQ5)
Execution time and energy consumption are correlated, and the strength of this relation differs between languages and platforms.

TODO: Decide whether language and platform main effects (RQ2/RQ3) get their own hypotheses. Note: the language main effect is largely expected from prior work [10]; the novelty lies in the interactions.

TODO: Pre-register hypotheses and analysis plan (e.g., OSF) before data collection.

5 Experimental Setup

5.1 Study Design

We use a full factorial design with the independent variables *LLM* ([TBD] levels), *programming language* (Python, Java, C++), and *hardware platform* (PC, server, Raspberry Pi). Programming tasks are treated as a random factor. The dependent variables are energy consumption (J, primary), execution time, peak memory usage, and functional correctness.

5.2 Large Language Models

TODO: Select models. Criteria: mix of proprietary and open-weight models, different sizes/vendors, availability of a fixed model version/snapshot for reproducibility. Record exact model identifiers, access dates, and decoding parameters.

Table 2: LLMs under study.

Model	Vendor	Version / snapshot	Access
[TBD]	[TBD]	[TBD]	API / local

5.3 Programming Tasks

TODO: Select 10–15 tasks covering sorting, searching, numerical computation, string processing, recursion, data structures, and memory-intensive workloads. Requirements: (i) language-independent specification, (ii) a test suite for correctness, (iii) a scalable input size so that each run lasts long enough for reliable energy measurement (target: $\geq$[TBD] s per run), (iv) low risk of training-data contamination. Candidate sources: Computer Language Benchmarks Game, EvoEval, EvalPerf, adapted LeetCode/Rosetta Code problems.

5.4 Hardware Platforms and Software Environment

Table 3: Hardware and software configuration.

	Desktop PC	Server	Raspberry Pi
CPU / architecture	[TBD]	[TBD]	[TBD] (ARM)
RAM	[TBD]	[TBD]	[TBD]
OS / kernel	[TBD]	[TBD]	[TBD]
Python (interpreter)	[TBD]	[TBD]	[TBD]
Java (JDK / JVM)	[TBD]	[TBD]	[TBD]
C++ compiler, flags	[TBD]	[TBD]	[TBD]
Energy measurement	[TBD]	[TBD]	[TBD]

TODO: Use identical language-runtime versions on all platforms where possible (e.g., via a pinned container image or toolchain). Fix CPU frequency governor, disable turbo/boost where feasible, and document all deviations.

5.5 Energy Measurement

TODO: Decide measurement instruments per platform. Key issue: RAPL (PC, server) measures CPU package/DRAM, whereas an external meter (Raspberry Pi) measures whole-board power. Options: (a) use external whole-system meters on all three platforms for comparability, optionally complemented by RAPL; (b) compare rankings *within* each platform rather than absolute Joules *across* platforms. Subtract idle baseline power.

6 Methodology and Data Collection

6.1 Prompting and Code Generation

TODO: Define the prompt template; it differs between languages only in the target language and the required I/O interface. Specify: zero-shot, no energy-related hints (baseline), temperature, number of samples per task/LLM/language (k), handling of non-compiling outputs (no manual repair, or a fixed number of automated retries).

6.2 Functional Correctness

TODO: Every generated program is executed against the task's test suite. Only programs passing all tests enter the energy analysis. Correctness rates are reported separately per LLM and language (**RQ1**, **RQ2**).

6.3 Measurement Procedure

TODO: Describe per run: warm-up, number of repetitions (n), randomised execution order, cool-down / idle intervals, idle-power baseline, JVM warm-up policy, process isolation, minimal background services. Record ambient temperature if possible.

6.4 Statistical Analysis

TODO: Planned analysis (to be finalised before data collection): (i) descriptive statistics (median, IQR) per cell; (ii) linear mixed-effects model on log(energy) with fixed effects LLM, language, platform and all interactions, and random intercepts for task (and generated solution); (iii) Type-III tests / likelihood-ratio tests for interactions, effect sizes, post-hoc comparisons with multiplicity correction; (iv) rank stability across languages and platforms via Kendall's τ / Friedman test (**RQ4**); (v) Spearman correlation between energy, time, and memory (**RQ5**). Check model assumptions; fall back to non-parametric or aligned-rank-transform methods if violated.

7 Results

7.1 Functional Correctness

TODO: Correctness rates per LLM $\times$ language.

7.2 RQ1: Differences between LLMs

TODO:

7.3 RQ2: Differences between Programming Languages

TODO:

7.4 RQ3: Differences between Hardware Platforms

TODO:

7.5 RQ4: Interaction Effects and Rank Stability

TODO:

7.6 RQ5: Relation of Energy, Time, and Memory

TODO:

8 Discussion

TODO: Interpret results with respect to the RQs; relate to prior work [1, 6, 10, 15]; analyse *why* differences arise (e.g., algorithmic choices, library usage, data structures) by qualitatively inspecting selected solutions; implications for developers and for LLM benchmarking.

9 Threats to Validity

Construct validity. Energy measurement methods differ in which components they capture (CPU package and DRAM via RAPL vs. whole-board power via external meters), which limits the comparability of absolute values across platforms.

TODO: Describe mitigation chosen in Section 5.

Internal validity. Measurements may be influenced by background processes, thermal state, frequency scaling, JIT warm-up, and garbage collection. LLM outputs are non-deterministic.

TODO: Mitigations: repetitions, randomised order, fixed governor, multiple samples per task.

External validity. Results are limited to the selected LLMs (and their versions at the time of the study), tasks, three languages, and three platforms, and may not generalise to other models, languages, or architectures, or to larger, real-world codebases.

Conclusion validity.

TODO: Multiple testing, assumption violations, sample size / power.

Confounding of model and implementation. The effect of an LLM cannot be fully separated from the concrete algorithm it chooses; these strategy differences are, however, part of the phenomenon under study.

Data contamination. Benchmark tasks and reference solutions may be part of the LLMs' training data.

10 Conclusion and Future Work

TODO: Summarise answers to RQ1–RQ5 once results are available.

Future work.

TODO: E.g., energy-aware prompting, additional languages (Rust, Go, JavaScript), GPU workloads, larger real-world tasks, energy cost of generation vs. execution.

Data availability.

TODO: Link to replication package (tasks, prompts, generated code, scripts, raw data).

References

[1] Radu Apsan, Vincenzo Stoico, Michel Albonico, Rudra Dhar, Karthik Vaidhyanathan, and Ivano Malavolta. Generating energy-efficient code via large-language models – where are we now? In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering (ICSE)*, pages 426–439. ACM, 2026. doi: 10.1145/3744916.3764522. Preprint: arXiv:2509.10099.

[2] Humza Ashraf, Syed Muhammad Danish, Aris Leivadeas, Yazan Otoum, and Zeeshan Sattar. Energy-aware code generation with LLMs: Benchmarking small vs. large language models for sustainable AI programming. arXiv preprint arXiv:2508.08332, 2025. URL `https://arxiv.org/abs/2508.08332`.

[3] Tom Cappendijk, Pepijn de Reus, and Ana Oprescu. An exploration of prompting LLMs to generate energy-efficient code. In *Proceedings of the IEEE/ACM 9th International Workshop on Green and Sustainable Software (GREENS)*, pages 31–38. IEEE, 2025. doi: 10.1109/GREENS66463.2025.00010.

[4] Vlad-Andrei Cursaru, Laura Duits, Joel Milligan, Damla Ural, Berta Rodriguez Sanchez, Vincenzo Stoico, and Ivano Malavolta. A controlled experiment on the energy efficiency of the source code generated by Code Llama. arXiv preprint arXiv:2405.03616, 2024. URL `https://arxiv.org/abs/2405.03616`.

[5] Dong Huang, Yuhao Qing, Weiyi Shang, Heming Cui, and Jie M. Zhang. EffiBench: Benchmarking the efficiency of automatically generated code. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2402.02037.

[6] Md Arman Islam, Devi Varaprasad Jonnala, Ritika Rekhi, Pratik Pokharel, Siddharth Cilamkoti, Asif Imran, Tevfik Kosar, and Bekir Turkkan. Evaluating the energy-efficiency of the code generated by LLMs. arXiv preprint arXiv:2505.20324, 2025. URL `https://arxiv.org/abs/2505.20324`.

[7] Jiawei Liu, Songrun Xie, Junhao Wang, Yuxiang Wei, Yifeng Ding, and Lingming Zhang. Evaluating language models for efficient code generation. arXiv preprint arXiv:2408.06450, 2024. URL `https://arxiv.org/abs/2408.06450`.

[8] Sabiya Banu Masthan Ali, Syed Muhammad Danish, Aroosa Hameed, Oussema Kirmani, and Gautam Srivastava. Energy-efficient code generation using large language models: A systematic literature review. arXiv preprint arXiv:2603.00989, 2026. URL `https://arxiv.org/abs/2603.00989`.

[9] Yun Peng, Jun Wan, Yichen Li, and Xiaoxue Ren. COFFE: A code efficiency benchmark for code generation. arXiv preprint arXiv:2502.02827. Accepted at FSE 2025, 2025. URL `https://arxiv.org/abs/2502.02827`.

[10] Rui Pereira, Marco Couto, Francisco Ribeiro, Rui Rua, Jácome Cunha, João Paulo Fernandes, and João Saraiva. Energy efficiency across programming languages: How do energy, time, and memory relate? In *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*, pages 256–267, New York, NY, USA, 2017. ACM. doi: 10.1145/3136014.3136031.

[11] Rui Pereira, Marco Couto, Francisco Ribeiro, Rui Rua, Jácome Cunha, João Paulo Fernandes, and João Saraiva. Ranking programming languages by energy efficiency. *Science of Computer Programming*, 205:102609, 2021. doi: 10.1016/j.scico.2021.102609.

[12] Yuhao Qing, Boyu Zhu, Mingzhe Du, Zhijiang Guo, Terry Yue Zhuo, Qianru Zhang, Jie M. Zhang, Heming Cui, Siu-Ming Yiu, Dong Huang, See-Kiong Ng, and Luu Anh Tuan. EffiBench-X: A multi-language benchmark for measuring efficiency of LLM-generated code. arXiv preprint arXiv:2505.13004, 2025. URL `https://arxiv.org/abs/2505.13004`.

[13] Ruizhong Qiu, Weiliang Will Zeng, James Ezick, Christopher Lott, and Hanghang Tong. How efficient is LLM-generated code? A rigorous & high-standard benchmark. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2406.06647.

[14] Ritika Rekhi, Bing Zhang, Md Arman Islam, Jaya Krishna Pasham, Yeswanth Chitturi, Akshay Paramesha, Isha Valiveti, Asif Imran, Bekir Turkkan, and Tevfik Kosar. Improving the energy-efficiency of the code generated by LLMs through effective prompting. arXiv preprint arXiv:2610.02571, 2026. URL `https://arxiv.org/abs/2610.02571`.

[15] Lola Solovyeva, Sophie Weidmann, and Fernando Castor. AI-powered, but power-hungry? Energy efficiency of LLM-generated code. arXiv preprint arXiv:2502.02412, 2025. URL `https://arxiv.org/abs/2502.02412`.

[16] Tina Vartziotis, Ippolyti Dellatolas, George Dasoulas, Maximilian Schmidt, Florian Schneider, Tim Hoffmann, Sotirios Kotsopoulos, and Michael Keckeisen. Learn to code sustainably: An empirical study on LLM-based green code generation. arXiv preprint arXiv:2403.03344, 2024. URL `https://arxiv.org/abs/2403.03344`.

Open TODOs

- ☐ **TODO:** Translate and condense the existing German abstract (~300 words) to ~200 words. Final numbers and findings can only be added after the experiments; until then, the abstract describes goal and method only. 1
- ☐ **TODO:** Results and implications. 1
- ☐ **TODO:** Add 1–2 citations on the ICT/software share of energy consumption and on the adoption of AI coding assistants. 1
- ☐ **TODO:** Paper outline paragraph ("The remainder of this paper is structured as follows…") once the structure is final. 2
- ☐ **TODO:** Short overview: how LLMs generate code, common benchmarks (HumanEval, MBPP, EvalPlus, EvoEval, LeetCode-based sets), pass@k as correctness metric. Cite originals. 2
- ☐ **TODO:** Energy vs. power ($E = \int P\,dt$); software-based estimation (Intel/AMD RAPL, tools building on it such as Scaphandre, CodeCarbon, perf) vs. hardware-based measurement (external power meters, INA219/INA226 shunt sensors). Explain which components each method captures (package, DRAM, whole system) and why RAPL is not available on ARM-based Raspberry Pi boards. 2
- ☐ **TODO:** Summarise key methodological points relevant to our study (compiled vs. virtual-machine vs. interpreted languages; faster is not always more energy-efficient) – in own words, without anticipating our results. 2
- ☐ **TODO:** Verify Cappendijk et al. platform/tool and Cursaru et al. statistics from full texts. Re-run the arXiv/Scopus search shortly before submission. 4
- ☐ **TODO:** Decide whether language and platform main effects (RQ2/RQ3) get their own hypotheses. Note: the language main effect is largely expected from prior work [10]; the novelty lies in the interactions. 5
- ☐ **TODO:** Pre-register hypotheses and analysis plan (e.g., OSF) before data collection. . 5
- ☐ **TODO:** Select models. Criteria: mix of proprietary and open-weight models, different sizes/vendors, availability of a fixed model version/snapshot for reproducibility. Record exact model identifiers, access dates, and decoding parameters. 5
- ☐ **TODO:** Select 10–15 tasks covering sorting, searching, numerical computation, string processing, recursion, data structures, and memory-intensive workloads. Requirements: (i) language-independent specification, (ii) a test suite for correctness, (iii) a scalable input size so that each run lasts long enough for reliable energy measurement (target: $\geq$[TBD] s per run), (iv) low risk of training-data contamination. Candidate sources: Computer Language Benchmarks Game, EvoEval, EvalPerf, adapted LeetCode/Rosetta Code problems. 5
- ☐ **TODO:** Use identical language-runtime versions on all platforms where possible (e.g., via a pinned container image or toolchain). Fix CPU frequency governor, disable turbo/boost where feasible, and document all deviations. 5
- ☐ **TODO:** Decide measurement instruments per platform. Key issue: RAPL (PC, server) measures CPU package/DRAM, whereas an external meter (Raspberry Pi) measures whole-board power. Options: (a) use external whole-system meters on all three platforms for comparability, optionally complemented by RAPL; (b) compare rankings *within* each platform rather than absolute Joules *across* platforms. Subtract idle baseline power. 6
- ☐ **TODO:** Define the prompt template; it differs between languages only in the target language and the required I/O interface. Specify: zero-shot, no energy-related hints (baseline), temperature, number of samples per task/LLM/language (k), handling of non-compiling outputs (no manual repair, or a fixed number of automated retries). . 6

- ☐ **TODO:** Every generated program is executed against the task's test suite. Only programs passing all tests enter the energy analysis. Correctness rates are reported separately per LLM and language (**RQ1**, **RQ2**). 6
- ☐ **TODO:** Describe per run: warm-up, number of repetitions (n), randomised execution order, cool-down / idle intervals, idle-power baseline, JVM warm-up policy, process isolation, minimal background services. Record ambient temperature if possible. . . 6
- ☐ **TODO:** Planned analysis (to be finalised before data collection): (i) descriptive statistics (median, IQR) per cell; (ii) linear mixed-effects model on log(energy) with fixed effects LLM, language, platform and all interactions, and random intercepts for task (and generated solution); (iii) Type-III tests / likelihood-ratio tests for interactions, effect sizes, post-hoc comparisons with multiplicity correction; (iv) rank stability across languages and platforms via Kendall's τ / Friedman test (**RQ4**); (v) Spearman correlation between energy, time, and memory (**RQ5**). Check model assumptions; fall back to non-parametric or aligned-rank-transform methods if violated. 6
- ☐ **TODO:** Correctness rates per LLM × language. 6
- ☐ **TODO:** 6
- ☐ **TODO:** 6
- ☐ **TODO:** 6
- ☐ **TODO:** 7
- ☐ **TODO:** 7
- ☐ **TODO:** Interpret results with respect to the RQs; relate to prior work [1, 6, 10, 15]; analyse *why* differences arise (e.g., algorithmic choices, library usage, data structures) by qualitatively inspecting selected solutions; implications for developers and for LLM benchmarking. 7
- ☐ **TODO:** Describe mitigation chosen in Section 5. 7
- ☐ **TODO:** Mitigations: repetitions, randomised order, fixed governor, multiple samples per task. 7
- ☐ **TODO:** Multiple testing, assumption violations, sample size / power. 7
- ☐ **TODO:** Summarise answers to RQ1–RQ5 once results are available. 7
- ☐ **TODO:** E.g., energy-aware prompting, additional languages (Rust, Go, JavaScript), GPU workloads, larger real-world tasks, energy cost of generation vs. execution. . . 7
- ☐ **TODO:** Link to replication package (tasks, prompts, generated code, scripts, raw data). 8